Experiment 2

Aim: Implement Multi Regression, Lasso, and Ridge Regression on real-world datasets

Dataset Source:
Concrete Compressive Strength:
https://archive.ics.uci.edu/dataset/165/concrete+compressive+strength

Cement (component 1)(kg in a m^3 mixture)	Blast Furnace Slag (component 2)(kg in a m^3 mixture)	Fly Ash (component 3)(kg in a m^3 mixture)	Water (component 4)(kg in a m^3 mixture)	Superplasticizer (component 5)(kg in a m^3 mixture)	Coarse Aggregate (component 6)(kg in a m^3 mixture)	Fine Aggregate (component 7)(kg in a m^3 mixture)	Age (day)	Concrete compressive strength(MPa, megapascals)
540.0	0.0	0.0	162.0	2.5	1040.0	676.0	28	79.99
540.0	0.0	0.0	162.0	2.5	1055.0	676.0	28	61.89
332.5	142.5	0.0	228.0	0.0	932.0	594.0	270	40.27
332.5	142.5	0.0	228.0	0.0	932.0	594.0	365	41.05
198.6	132.4	0.0	192.0	0.0	978.4	825.5	360	44.30
266.0	114.0	0.0	228.0	0.0	932.0	670.0	90	47.03
380.0	95.0	0.0	228.0	0.0	932.0	594.0	365	43.70
380.0	95.0	0.0	228.0	0.0	932.0	594.0	28	36.45
266.0	114.0	0.0	228.0	0.0	932.0	670.0	28	45.85

Dataset Description:

Dataset Overview

- **Instances:** 1030 real concrete mixtures
- **Features:** 8 numerical input variables
- **Target:** Concrete compressive strength (MPa)

Input Features

Feature	Real-Life Meaning
Cement	Cement quantity (kg/m³)
Blast Furnace Slag	Industrial waste material
Fly Ash	By-product from power plants
Water	Mixing water
Superplasticizer	Chemical for workability

Coarse Aggregate	Gravel
Fine Aggregate	Sand
Age	Days after pouring

Target Variable:

- Concrete compressive strength (MPa)

Mathematical Formulation of the Algorithm:
1.Multiple Linear Regression

$$\hat{y} = \beta_0 + \sum_{i=1}^{n} \beta_i x_i$$

2. Ridge Regression (L2 Regularization)

$$J(\beta) = \sum (y_i - \hat{y}_i)^2 + \alpha \sum \beta_i^2$$

3. Lasso Regression (L1 Regularization)

$$J(\beta) = \sum (y_i - \hat{y}_i)^2 + \alpha \sum |\beta_i|$$

Algorithm Limitations:

Multiple Linear Regression

- Sensitive to multicollinearity
- Less accurate for complex material interactions

Ridge Regression

- Cannot fully eliminate irrelevant inputs
- Needs tuning of α

Lasso Regression

- May drop important materials if α is too high
- Struggles with highly correlated inputs

Methodology / Workflow:

1. Concrete Mix Data
2. Data Cleaning & Standardization
3. Train–Test Split
4. Model Training (LR, Ridge, Lasso)
5. Hyperparameter Tuning
6. Performance Evaluation
7. Model Comparison

Linear Regression Workflow

1. Upload Dataset (Colab)

```
df = pd.read_excel("Concrete_Data.xls")
df.head()
```

	Cement (component 1) (kg in a m^3 mixture)	Blast Furnace Slag (component 2) (kg in a m^3 mixture)	Fly Ash (component 3) (kg in a m^3 mixture)	Water (component 4) (kg in a m^3 mixture)	Superplasticizer (component 5)(kg in a m^3 mixture)	Coarse Aggregate (component 6) (kg in a m^3 mixture)	Fine Aggregate (component 7) (kg in a m^3 mixture)	Age (day)	Concrete compressive strength(MPa, megapascals)
0	540.0	0.0	0.0	162.0	2.5	1040.0	676.0	28	79.986111
1	540.0	0.0	0.0	162.0	2.5	1055.0	676.0	28	61.887366
2	332.5	142.5	0.0	228.0	0.0	932.0	594.0	270	40.269535
3	332.5	142.5	0.0	228.0	0.0	932.0	594.0	365	41.052780
4	198.6	132.4	0.0	192.0	0.0	978.4	825.5	360	44.296075

2. Dataset Overview

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1030 entries, 0 to 1029
Data columns (total 9 columns):
 #   Column                                             Non-Null Count  Dtype
---  ------                                             --------------  -----
 0   Cement (component 1)(kg in a m^3 mixture)          1030 non-null   float64
 1   Blast Furnace Slag (component 2)(kg in a m^3 mixture)  1030 non-null   float64
 2   Fly Ash (component 3)(kg in a m^3 mixture)         1030 non-null   float64
 3   Water  (component 4)(kg in a m^3 mixture)          1030 non-null   float64
 4   Superplasticizer (component 5)(kg in a m^3 mixture)  1030 non-null   float64
 5   Coarse Aggregate  (component 6)(kg in a m^3 mixture)  1030 non-null   float64
 6   Fine Aggregate (component 7)(kg in a m^3 mixture)  1030 non-null   float64
 7   Age (day)                                          1030 non-null   int64
 8   Concrete compressive strength(MPa, megapascals)    1030 non-null   float64
dtypes: float64(8), int64(1)
memory usage: 72.6 KB
```

	Cement (component 1)(kg in a m^3 mixture)	Blast Furnace Slag (component 2) (kg in a m^3 mixture)	Fly Ash (component 3)(kg in a m^3 mixture)	Water (component 4)(kg in a m^3 mixture)	Superplasticizer (component 5)(kg in a m^3 mixture)	Coarse Aggregate (component 6) (kg in a m^3 mixture)	Fine Aggregate (component 7) (kg in a m^3 mixture)	Age (day)	Concrete compressive strength(MPa, megapascals)
count	1030.000000	1030.000000	1030.000000	1030.000000	1030.000000	1030.000000	1030.000000	1030.000000	1030.000000
mean	281.165631	73.895485	54.187136	181.566359	6.203112	972.918592	773.578883	45.662136	35.817836
std	104.507142	86.279104	63.996469	21.355567	5.973492	77.753818	80.175427	63.169912	16.705679
min	102.000000	0.000000	0.000000	121.750000	0.000000	801.000000	594.000000	1.000000	2.331808
25%	192.375000	0.000000	0.000000	164.900000	0.000000	932.000000	730.950000	7.000000	23.707115
50%	272.900000	22.000000	0.000000	185.000000	6.350000	968.000000	779.510000	28.000000	34.442774
75%	350.000000	142.950000	118.270000	192.000000	10.160000	1029.400000	824.000000	56.000000	46.136287
max	540.000000	359.400000	200.100000	247.000000	32.200000	1145.000000	992.600000	365.000000	82.599225

3. Separate Features & Target

```
# Target variable
target_column = 'Concrete compressive strength(MPa, megapascals) '

X = df.drop(target_column, axis=1)
y = df[target_column]
```

4. Feature Scaling

```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

5. Split data into training and testing sets

```
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.2, random_state=42
)
```

MODEL 1: Multiple Linear Regression

```
lr = LinearRegression()
lr.fit(X_train, y_train)

lr_pred = lr.predict(X_test)

print("Multiple Linear Regression")
print("R2 Score:", r2_score(y_test, lr_pred))
print("RMSE:", np.sqrt(mean_squared_error(y_test, lr_pred)))
```

```
Multiple Linear Regression
R2 Score: 0.6275416055429017
RMSE: 9.79670783239845
```

MODEL 2: Ridge Regression (With Hyperparameter Tuning)

```
Ridge Regression
Best Alpha: {'alpha': 1}
R2 Score: 0.6275703720172157
RMSE: 9.796329505328783
```

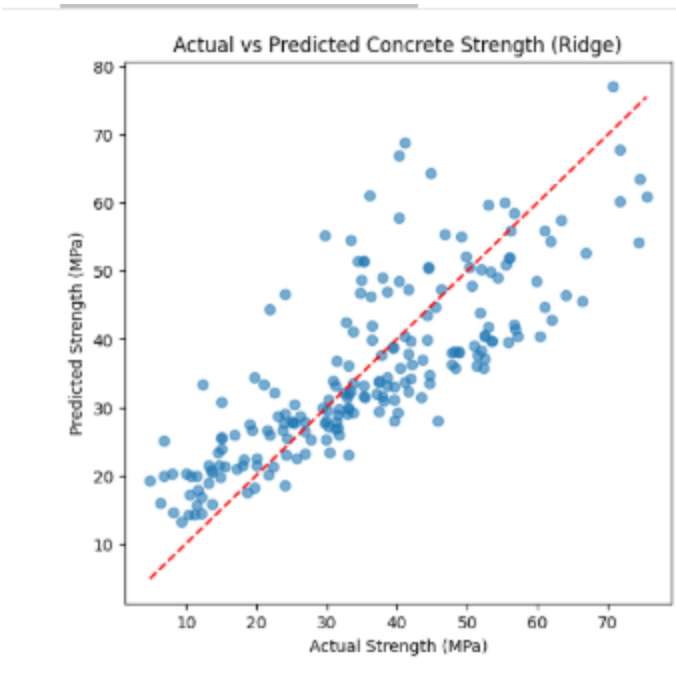
MODEL 3: Lasso Regression (With Hyperparameter Tuning)

```
ridge = Ridge()

ridge_params = {
    'alpha': [0.1, 1, 10, 50, 100]
}

ridge_cv = GridSearchCV(
    ridge,
    ridge_params,
    cv=5,
    scoring='r2'
)

ridge_cv.fit(X_train, y_train)

ridge_pred = ridge_cv.predict(X_test)

print("Ridge Regression")
print("Best Alpha:", ridge_cv.best_params_)
print("R2 Score:", r2_score(y_test, ridge_pred))
print("RMSE:", np.sqrt(mean_squared_error(y_test, ridge_pred)))
```

```python
lasso_params = {
    'alpha': [0.01, 0.1, 1, 10]
}

lasso_cv = GridSearchCV(
    lasso,
    lasso_params,
    cv=5,
    scoring='r2'
)

lasso_cv.fit(X_train, y_train)

lasso_pred = lasso_cv.predict(X_test)

print("Lasso Regression")
print("Best Alpha:", lasso_cv.best_params_)
print("R2 Score:", r2_score(y_test, lasso_pred))
print("RMSE:", np.sqrt(mean_squared_error(y_test, lasso_pred)))
```

```
Lasso Regression
Best Alpha: {'alpha': 0.01}
R2 Score: 0.6275470303917419
RMSE: 9.796636487714814
```

	Model	R2 Score	RMSE
0	Linear Regression	0.627542	9.796708
1	Ridge Regression	0.627570	9.796330
2	Lasso Regression	0.627547	9.796636



Performance Analysis:

Evaluation Metrics

- **R^2 Score** → Strength prediction accuracy
- **RMSE** → Error in MPa (engineering relevance)

Interpretation

- Linear Regression: baseline prediction
- Ridge: more stable predictions
- Lasso: highlights key ingredients affecting strength

Hyperparameter Tuning:

Tuned Parameter

- **α (alpha)** for Ridge and Lasso

Technique

- GridSearchCV with 5-fold cross-validation

Real Impact

- Prevents overfitting
- Improves generalization to unseen concrete mixes

SDG No.	SDG Goal Name	Relevance to the Project
SDG 9	Industry, Innovation and Infrastructure	The project applies machine learning to predict concrete compressive strength, supporting the development of safe, reliable, and sustainable infrastructure while promoting innovation in construction technology.
SDG 11	Sustainable Cities and Communities	Accurate prediction of concrete strength ensures safer buildings and infrastructure, contributing to resilient urban development and improved living conditions.
SDG 12	Responsible Consumption and Production	The model helps optimize material usage in concrete mixtures, reducing waste, minimizing overuse of raw materials, and lowering environmental impact.